

Thème:
**MISE EN PLACE D'UN RUNTIME POUR L'EXECUTION DE
CALCULS SUR MACHINES VOLONTAIRES DANS UN
ENVIRONNEMENT A RESSOURCES LIMITÉES**

Rapport de travail hebdomadaire
Par NGNAPA NGOULE Ashley

Département d'informatique, Université de Yaoundé 1

28 Novembre 2025

Sommaire

- ① Les tâches de la semaine
- ② An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment
- ③ Explication du mécanisme de fonctionnement des méthodes d'isolation sous Linux
 - Exemple concret : isolation UTS
- ④ Les tâches à faire

Les tâches de la semaine

Les tâches à faire

Le travail de cette semaine s'est articulé autour des points suivants :

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment"
- L'implémentation de fonctionnalités sur l'application volontaire
 - Ajout de la gestion des clients
 - Ajout de la validation des clients

Les tâches à faire

Le travail de cette semaine s'est articulé autour des points suivants :

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment"
- L'implémentation de fonctionnalités sur l'application volontaire
 - Ajout de la gestion des clients
 - Ajout de la validation des clients

Les tâches à faire

Le travail de cette semaine s'est articulé autour des points suivants :

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment"
- L'implémentation de fonctionnalités sur l'application volontaire
 - Ajout de la gestion des clients
 - Ajout de la validation des clients

An Ultra-lightweight Container that Maximizes Memory Sharing and Minimizes the Runtime Environment

Objectif et source de l'article

Objectif de l'article

Cet article de recherche se concentre sur la conception et l'implémentation d'un conteneur ultra-léger visant à maximiser le partage de la mémoire et à minimiser l'environnement d'exécution.

Source et année de publication

Rédigé par Liqing Zhang en 2019, publié dans le journal International Journal of Science.

Contexte et enjeux

L'article s'inscrit dans le contexte de l'essor de la technologie des conteneurs, qui a transformé les centres de données. L'article identifie des lacunes importantes de Docker, notamment la redondance du volume des images et le manque d'efficacité dans le partage des ressources. L'enjeu principal était de dépasser les limitations de Docker en matière de redondance de la mémoire, de stockage et de sécurité.

Question de recherche

La question de recherche est : comment concevoir et implémenter une nouvelle architecture de conteneur ultra-léger capable de maximiser le partage des ressources et de minimiser l'environnement d'exécution nécessaire ?

État de l'art

Les auteurs ont réalisé une série de lectures pour arriver à leurs fins :

- Comparaison VM vs conteneurs
- Travaux de Paul Merge et al. : existence des problèmes de redondance et manque d'efficacité
- Ferreira J. B. et Cello M. et al. : importance critique et impact du partage des bibliothèques, conception de REG pour maximiser le partage de la mémoire de l'hôte
- Concept Unikernel : abandon du concept d'image au profit d'une approche où l'application (le code) est l'image

État de l'art

Les auteurs ont réalisé une série de lectures pour arriver à leurs fins :

- Comparaison VM vs conteneurs
- Travaux de Paul Merge et al. : existence des problèmes de redondance et manque d'efficacité
- Ferreira J. B. et Cello M. et al. : importance critique et impact du partage des bibliothèques, conception de REG pour maximiser le partage de la mémoire de l'hôte
- Concept Unikernel : abandon du concept d'image au profit d'une approche où l'application (le code) est l'image

État de l'art

Les auteurs ont réalisé une série de lectures pour arriver à leurs fins :

- Comparaison VM vs conteneurs
- Travaux de Paul Merge et al. : existence des problèmes de redondance et manque d'efficacité
- Ferreira J. B. et Cello M. et al. : importance critique et impact du partage des librairies, conception de REG pour maximiser le partage de la mémoire de l'hôte
- Concept Unikernel : abandon du concept d'image au profit d'une approche où l'application (le code) est l'image

État de l'art

Les auteurs ont réalisé une série de lectures pour arriver à leurs fins :

- Comparaison VM vs conteneurs
- Travaux de Paul Merge et al. : existence des problèmes de redondance et manque d'efficacité
- Ferreira J. B. et Cello M. et al. : importance critique et impact du partage des bibliothèques, conception de REG pour maximiser le partage de la mémoire de l'hôte
- Concept Unikernel : abandon du concept d'image au profit d'une approche où l'application (le code) est l'image

Solution proposée

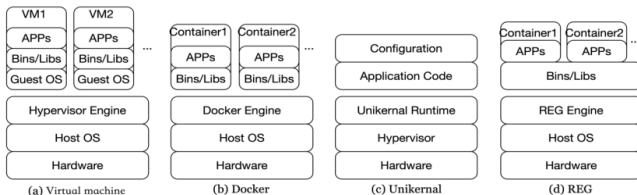
La solution proposée est un moteur de gestion de conteneurs appelé REG (Runtime Environment Generation) :

- Conçu sur une architecture client/serveur
- Objectif : maximiser le partage de la mémoire et minimiser la taille de l'environnement
- Granularité réduite

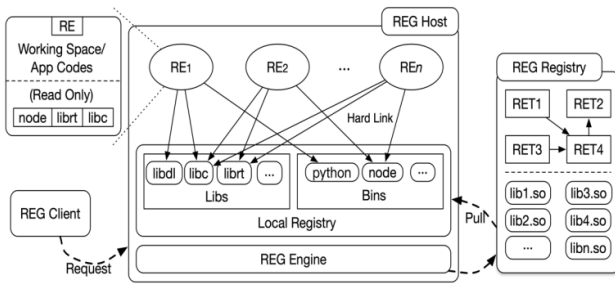
Architecture

- Conception client/serveur
- Utilise les mécanismes de Namespaces et Cgroups pour l'isolation des ressources
- Élimine le concept d'image
- Dispose de REG Register, Engine et Client

Architecture comparée



Architecture REG



Les étapes de génération de REG

- Génération de la structure de fichiers
- Obtention du runtime
- Liaison du runtime avec le point de montage créé

Benchmarks, résultats et commentaires

Comparaison de REG avec Docker sur 5 logiciels : Python, Node.js, MySQL, Nginx, Tomcat

- **Volume de l'image** : réduction moyenne de 18% par rapport à Docker
- **Temps de démarrage** : REG 373 ms vs Docker 405 ms (7,9% plus rapide) ; en tempête de démarrage REG 405 ms vs Docker 827 ms (51% plus rapide)
- **Empreinte mémoire** : réduction moyenne de 47,6% par rapport à Docker

Appréciation générale et point de vue

Ce que je retiens de cet article :

- Maximiser le partage de mémoire via les hard links
- Pas d'image fixe : un fichier définit le contenu du runtime
- Modèle de templates de conteneurs
- Architecture client (manager), Registry distante, Engine (construction de l'image)

Explication du mécanisme de fonctionnement des méthodes d'isolation sous Linux

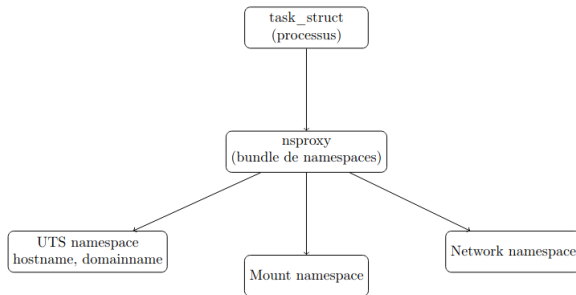
Isolation des processus sous Linux

`fork()` : crée un processus enfant quasi-identique au parent. `clone()` : contrôle fin des ressources partagées ou isolées. `unshare()` : détache un processus existant de certains contextes.

- `clone()` : création de processus/threads avec flags (ex : `CLONE_NEWPID`, `CLONE_NEWNET`, `CLONE_NEWNS`, `CLONE_NEWUTS`, `CLONE_NEWIPC`)
- `unshare()` : modifie l'isolation du processus courant, aucun nouveau processus créé

Structures de données impliquées

- **Nsproxy** : regroupe tous les namespaces d'un processus
 - **Appels système** : `clone`, `unshare`, `setns`
 - **Rôle du noyau** : créer/détacher/attacher des namespaces via `nsproxy.c`
- ❶ `unshare(CLONE_NEWUTS)` ;
 - ❷ `sethostname("containerA", strlen("containerA"))` ;
 - ❸ La `task_struct->nsproxy->uts_ns` pointe vers le nouvel UTS namespace



Les tâches à faire

Les tâches à faire pour la semaine prochaine

- Terminer la lecture et le résumé de l'article "Singularity"
- Proposer une architecture du runtime à créer
- Étudier l'isolation sous Windows

Les tâches à faire pour la semaine prochaine

- Terminer la lecture et le résumé de l'article "Singularity"
- Proposer une architecture du runtime à créer
- Étudier l'isolation sous Windows

Les tâches à faire pour la semaine prochaine

- Terminer la lecture et le résumé de l'article "Singularity"
- Proposer une architecture du runtime à créer
- Étudier l'isolation sous Windows

Merci !!!